

Arquitectura de Autenticación y Notificaciones

ECOSISTEMA EFICIENTE PARA PLATAFORMA DE BARBERÍAS

Este documento técnico define la estructura, implementación y políticas de seguridad necesarias para desplegar un sistema de gestión de barberías utilizando **Supabase Auth** como motor de identidad y la **Web Push API** para el envío automatizado de notificaciones push de forma 100% gratuita.

Ventaja Estratégica: Al unificar la autenticación nativa de Supabase con el estándar de notificaciones Push del navegador, eliminamos la dependencia de APIs de mensajería costosas o librerías de simulación web que conllevan riesgos de baneo o bloqueos telefónicos por parte de Meta.

1. Autenticación Centralizada con Supabase Auth

Supabase Auth nos provee un backend de identidad completo basado en tokens JWT (JSON Web Tokens) que se conecta de manera nativa con las políticas de base de datos. Para maximizar la conversión y simplificar el flujo al cliente final, implementaremos el inicio de sesión único (SSO) mediante **Google OAuth**.

La siguiente tabla muestra la matriz de accesos y roles estructurada dentro de nuestra base de datos:

Rol del Usuario	Método de Autenticación	Alcance del Permiso (RLS)
Cliente	Google OAuth / Magic Link	Lectura y escritura exclusiva de sus propias citas y suscripciones de notificación.
Barbero	Email / Contraseña	Lectura de la agenda global del local y actualización de estados de servicios asignados.
Administrador	Email / Contraseña + MFA	Acceso total a métricas de negocio, inventario, comisiones y configuración de locales.

A continuación se detalla la inicialización del cliente de Supabase y la función encargada de disparar el flujo de autenticación federada con Google desde el entorno cliente:

```
// lib/supabaseClient.js
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;
const supabaseAnonKey = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;

export const supabase = createClient(supabaseUrl, supabaseAnonKey);

// auth/googleAuth.js
export async function iniciarSesionConGoogle() {
  const { data, error } = await supabase.auth.signInWithOAuth({
    provider: 'google',
    options: {
      redirectTo: `${window.location.origin}/dashboard/citas`,
    },
  });
}

if (error) throw new Error(error.message);
return data;
}
```

2. Canal de Notificaciones con Web Push API

La Web Push API permite enviar mensajes dirigidos desde el servidor web hacia los dispositivos de los usuarios utilizando los servidores de mensajería del navegador (FCM para Chrome/Android y APNs para Safari/iOS). Requiere de un script persistente llamado `Service Worker` corriendo en segundo plano.

El Service Worker es el encargado de escuchar el evento de red, decodificar los datos criptográficos y renderizar la alerta nativa directamente en la pantalla de bloqueo o centro de notificaciones:

```
// public/sw.js (Service Worker ejecutable en segundo plano)
self.addEventListener('push', function(event) {
  const payload = event.data ? event.data.json() : { title: 'Barbería', body: 'Actualización' };

  const options = {
    body: payload.body,
    icon: '/icons/logo-192.png',
    badge: '/icons/badge-72.png',
    vibrate: [100, 50, 100],
    data: { url: payload.url || '/dashboard/citas' }
  };

  event.waitUntil(
    self.registration.showNotification(payload.title, options)
  );
});
```

Una vez que el usuario confirma el agendamiento de su cita, el frontend captura la clave pública VAPID provista por nuestro backend, genera la suscripción del navegador y la almacena en nuestro ecosistema de Supabase:

```
// utils/pushManager.js
export async function registrarSuscripcionPush(usuarioId) {
  const registration = await navigator.serviceWorker.ready;

  const subscription = await registration.pushManager.subscribe({
    userVisibleOnly: true,
    applicationServerKey: urlBase64ToUint8Array(process.env.NEXT_PUBLIC_VAPID_PUBLIC_KEY)
  });

  await supabase
    .from('suscripciones_push')
    .upsert({
      user_id: usuarioId,
      endpoint: subscription.endpoint,
      keys_p256dh: btoa(String.fromCharCode(...new Uint8Array(subscription.getKey('p256dh')))),
      keys_auth: btoa(String.fromCharCode(...new Uint8Array(subscription.getKey('auth'))))
    });
}
```

3. Modelo de Datos y Seguridad en Supabase (Esquema SQL)

Para asegurar que las claves criptográficas de los navegadores estén protegidas de lecturas maliciosas, es estrictamente obligatorio activar el aislamiento de filas mediante políticas **Row Level Security (RLS)** dentro de Supabase. El siguiente script SQL define la tabla y sus restricciones de acceso:

```
-- Creación de la tabla de persistencia de suscripciones
CREATE TABLE public.suscripciones_push (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
  endpoint TEXT NOT NULL UNIQUE,
  keys_p256dh TEXT NOT NULL,
  keys_auth TEXT NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text, now()) NOT NULL
);

-- Habilitación imperativa de Row Level Security (RLS)
ALTER TABLE public.suscripciones_push ENABLE ROW LEVEL SECURITY;

-- Declaración de políticas de aislamiento estricto
CREATE POLICY "Inserción restringida al titular de la sesión"
  ON public.suscripciones_push FOR INSERT
  WITH CHECK (auth.uid() = user_id);

CREATE POLICY "Lectura restringida al titular de la sesión"
  ON public.suscripciones_push FOR SELECT
  USING (auth.uid() = user_id);
```

4. Automatización Mediante Tareas Programadas (Vercel Crons)

El envío automático de recordatorios sin intervención del staff se gestiona declarando un **Cron Job** dentro de la configuración de Vercel. Este servicio invoca una función Serverless en un bloque horario determinado:

```
// vercel.json
{
  "crons": [
    {
      "path": "/api/cron/enviar-recordatorios",
      "schedule": "0 14 * * *"
    }
  ]
}
```

*Nota: El horario '0 14 * * *' se ejecuta de forma automática a las 08:00 AM hora local (equivalente a UTC-6), ideal para enviar la agenda del día.*

La lógica interna de la función Serverless procesará las colas de comunicación según las siguientes tres reglas operativas:

Evento Desencadenante	Destinatario	Estructura del Mensaje Informativo
Confirmación de Reserva	Cliente	"¡Cita confirmada! Te esperamos el [Fecha] a las [Hora] con el barbero [Nombre]."
Recordatorio Matutino	Cliente	"Hola [Cliente], te recordamos que tienes una cita de corte agendada para hoy a las [Hora]."
Notificación de Cancelación	Barbero	"Atención: El cliente ha cancelado el turno de las [Hora]. Bloque disponible en agenda."